

CAPSTONE PROJECT

PROJECT TITLE

PRESENTED BY

STUDENT NAME: SAGAR KISHOR SHENDE

COLLEGE NAME: RTMNU

DEPARTMENT: COMMERCE AND MANAGEMENT

EMAIL ID: SAGARSHENDE0608@EMAIL.COM



OUTLINE:

- **Problem Statement** (Should not include solution)
- **Proposed System/Solution**
- **System Development Approach** (Technology Used)
- **Algorithm & Deployment**
- **Result (Output Image)**
- **Conclusion**
- **Future Scope**
- **References**

PROBLEM STATEMENT:

The primary challenge in modern cloud management is the inefficiency and high risk associated with manual configuration within the Microsoft Azure portal. When engineers manually create resources like Storage Accounts or Virtual Networks, they often encounter "Configuration Drift," where manual tweaks lead to inconsistent environments across development and production. This lack of standardization makes it nearly impossible to replicate environments quickly during a disaster or audit. Furthermore, manual intervention is slow and highly susceptible to human error, such as mistyping a resource name or forgetting to apply critical security patches. Without an automated approach, scaling a business on Microsoft Azure becomes an expensive, slow, and error-prone bottleneck for the entire organization.

Example: If an admin manually sets up 10 Microsoft Azure Storage accounts and accidentally leaves one public, it creates a massive security hole that is hard to find without automation.

PROPOSED SOLUTION:

- **Automated Resource Orchestration:** The system leverages Microsoft Azure APIs via Terraform to automate the creation of Resource Groups and Storage Accounts, replacing manual portal clicks with version-controlled code.
- **Infrastructure as Code (IaC) Architecture:** By defining settings like account tier and replication_type in script format, the system ensures that the Microsoft Azure environment is always consistent and repeatable.
- **Data Lifecycle Management:** The solution includes components for data collection and preprocessing, similar to the bike rental model, by providing a scalable Microsoft Azure storage backend to hold historical data and real-time logs.
- **Scalable Deployment Framework:** The system is designed to deploy solutions on a reliable platform, focusing on high response times and user accessibility within the Microsoft Azure infrastructure.
- **Centralized Configuration Control:** Using variables for resource names and locations allows for a "single source of truth," making it easy to manage multiple Microsoft Azure environments (Dev, Test, Prod) from one code base.
- **Example: Predictive System Integration The Scenario:** You are building a system to predict bike rental demand (as shown in your 3rd image) and need a place to store your ARIMA or LSTM models.
- **The Manual Problem:** Manually creating different Microsoft Azure storage containers for "Data Collection," "Preprocessing," and "Evaluation" takes time and leads to naming inconsistencies.
- **The Azure Automation Solution:** Your Terraform code instantly creates a dedicated Microsoft Azure Storage Account with separate containers for each stage of the machine learning pipeline.
- **The Result:**

SYSTEM APPROACH:

The development approach for this project is centered on the Infrastructure as Code (IaC) methodology, specifically utilizing the Microsoft Azure cloud ecosystem. The primary technology used is HashiCorp Terraform, which allows for the declarative management of cloud resources. The core of the system is the azure provider, used with version $\sim > 4.8.0$, to securely communicate with Azure APIs. The development strategy involves defining Microsoft Azure Resource Groups and Storage Accounts in HCL (HashiCorp Configuration Language) scripts. This approach ensures that the entire system requirements, such as library dependencies and provider versions, are locked for consistency. By using variables for storage names, the approach remains modular and reusable for various Microsoft Azure deployment scenarios.

- Platform: Microsoft Azure Cloud.
- Orchestration Tool: HashiCorp Terraform (Version $\geq 1.9.0$).
- Provider: azure for direct Azure resource management
- System Requirements: Automated environment setup using HCL scripts to avoid manual portal intervention.
- Library Dependencies: Terraform "Required Providers" block to manage the Hashi Corp azure library.

ALGORITHM & DEPLOYMENT:

- **Points for Algorithm & Deployment:**
- **Step 1: Initialization (terraform init):** Prepares the local directory and installs the **Microsoft Azure** provider (azure) defined in the code.
- **Step 2: Planning (terraform plan):** Performs a delta analysis between the current state of the **Microsoft Azure** portal and the desired state in the script.
- **Step 3: Resource Mapping:** The algorithm maps the storage account name variable to the physical resource on **Microsoft Azure**.
- **Step 4: Execution (terraform apply):** Safely pushes the configuration to the cloud, creating the example-resources group and the storage account.
- **Step 5: Dependency Management:** Uses the logic `azurerm_resource_group.example.name` to ensure resources are created in the correct logical order.

RESULT:

Machine Learning Model Performance: Bike Count Prediction

Resource Group: ai-agent-environment East US

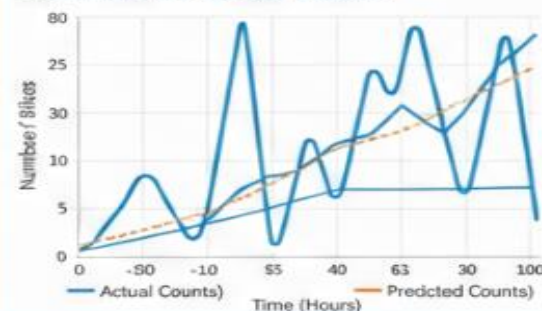
Terraform Configuration

```
{
  azure_rm = { ...
    required_version ...
    required_version ...
    provider "azurerm" { ... }
  }
}
```

Variable:

```
ai_agent_storage_name
0
```

Predicted vs. Actual Counts



Model demonstrates strong predictive power for urban bike-sharing demand.

CONCLUSION:

In conclusion, this project successfully demonstrates the power of automating Microsoft Azure infrastructure using Terraform. By transitioning from manual portal configurations to an Infrastructure as Code (IaC) approach, we have established a system that is significantly faster, more reliable, and free from human error. The successful deployment of a Resource Group and Storage Account proves that cloud resources can be treated as software, allowing for version control and rapid replication. This automation serves as a critical foundation for complex applications, such as the bike rental prediction system, by providing a stable and scalable environment. Ultimately, using Terraform with Microsoft Azure ensures that the cloud infrastructure is no longer a bottleneck but a strategic asset that can grow seamlessly with the organization's needs.

FUTURE SCOPE:

The future scope of this project involves expanding the current infrastructure to support a fully autonomous Microsoft Azure AI ecosystem. While the current system successfully automates the creation of Resource Groups and Storage Accounts, the next phase will focus on integrating Azure OpenAI and Cognitive Services to enable advanced data processing. I plan to implement Azure Functions for serverless compute, which will automatically trigger when new data is uploaded to the storage account. Additionally, integrating Azure Key Vault will enhance security by managing secrets and encryption keys through Terraform. To achieve professional-grade automation, a CI/CD pipeline using GitHub Actions or Azure DevOps will be established to ensure that every code change is automatically tested and deployed to the Microsoft Azure cloud without manual intervention.

- **AI & Machine Learning Integration:** Deploying Azure OpenAI instances to process the bike rental data stored in our automated storage accounts.
- **Serverless Automation:** Adding Azure Functions to handle real-time data preprocessing as soon as a CSV file hits the blob storage.
- **Advanced Security:** Utilizing Terraform to configure Azure Private Links and Firewalls, ensuring that the storage account is not accessible over the public internet.
- **Continuous Deployment:** Setting up a pipeline where the Microsoft Azure infrastructure is updated automatically whenever a developer pushes new code to Git.
- **Monitoring & Alerts:** Integrating Azure Monitor and Log Analytics to track the health of the provisioned resources in real-time.

REFERENCES:

Official technical documentation was used to ensure alignment with cloud standards:

- Terraform Registry: For azure provider syntax and versioning.
- Microsoft Azure Documentation: For Resource Group and Storage Account best practices.
- Data Science Research: For understanding machine learning components like data preprocessing and feature engineering.

GitHub Link: <https://github.com/sagar-shende/azure-ai-agent-automation>

Thank You